

Cache-Aware and GPU-Accelerated Charge Deposition for Electrostatic Particle-In-Cell Simulations

Elhadj Hocine
Independent Researcher
Email: hocineelhadj@gmail.com

Abstract—Charge deposition is a dominant cost in explicit Particle-In-Cell (PIC) plasma simulations because it performs irregular, scatter-oriented updates to a structured grid. This work presents a reproducible two-dimensional electrostatic PIC framework and evaluates Nearest Grid Point (NGP), Cloud-In-Cell (CIC), Triangular-Shaped Cloud (TSC), and Esirkepov charge-conserving deposition under cache-aware CPU optimizations. We compare Array-of-Structures (AoS) and Structure-of-Arrays (SoA) layouts, optional cell sorting, and OpenMP parallelization with thread-local grid privatization. Benchmarks measure sort and deposit time separately, effective memory bandwidth, roofline arithmetic intensity, charge conservation error, energy drift, a 2000-step conservation study, spectral grid noise, and CPU strong scaling on particle counts from 10^4 to 10^6 . Separate CPU results on Apple M1 Pro and GPU results on Tesla T4 (same-host CPU baseline) are reported to avoid misleading cross-platform comparisons. A Langmuir-wave validation confirms scheme-independent oscillation frequencies within 5% across all four deposition methods. On Tesla T4, atomic CUDA deposition achieves 5–8 $\times$ speedup over single-threaded host CPU at $N_p = 10^5$ – 10^6 ; multi-block privatized tiles preserve charge conservation but remain slower than atomics due to per-tile full-particle scans. Roofline analysis shows deposition is memory-bandwidth limited; CIC offers the best accuracy–performance trade-off on CPU, while GPU backends favor large N_p . Source code and archived CSVs are available at <https://github.com/elhadjhocine/pic-deposition> (Zenodo DOI: TODO-DOI).

Index Terms—Particle-In-Cell, charge deposition, high performance computing, cache optimization, plasma simulation

I. Introduction

Particle-In-Cell (PIC) methods combine Lagrangian macroparticles with Eulerian field representations and are widely used in kinetic plasma simulation [1]–[3]. Each explicit timestep performs charge deposition, field solve, field gather, and particle push. Among these stages, charge deposition is frequently a bottleneck because it scatters particle contributions into a grid with irregular memory access patterns [4]. Prior studies have shown that shape-function choice controls grid noise and stability [5], while charge-conserving deposition is essential for long-time accuracy [6], [7].

This work presents a performance-oriented evaluation of deposition algorithms and cache-aware CPU implementations in a reproducible 2D electrostatic PIC framework. This paper reports CPU microbenchmarks on Apple M1

Pro, GPU atomic deposition on Tesla T4, and shared physics validation. We study NGP, CIC, TSC, and Esirkepov charge-conserving deposition [6], along with particle sorting, AoS versus SoA layouts, and OpenMP parallelization with thread-local grid buffers. CUDA deposition kernels (atomics and multi-block privatized tiles) are benchmarked on Tesla T4 alongside CPU results on Apple M1 Pro.

Our contributions are:

- A reproducible 2D electrostatic PIC reference code with FFT-based Poisson solve and unified deposition interface.
- A systematic comparison of four deposition schemes with charge, energy, spectral-noise, and long-run conservation diagnostics.
- Cache-aware CPU study separating sort cost from deposit-kernel throughput for AoS/SoA layouts.
- Langmuir-wave validation, OpenMP privatization on Apple Silicon, and roofline-style GPU/CPU performance analysis.

II. Mathematical Background

The continuous charge density is

$$\rho(\mathbf{x}) = \sum_p q_p S(\mathbf{x} - \mathbf{x}_p), \quad (1)$$

where S is a compact shape function. Deposited charge must satisfy global conservation,

$$\int \rho(\mathbf{x}) dV = \sum_p q_p, \quad (2)$$

otherwise nonphysical fields and instability can arise [8]. We report conservation error normalized by $\sum_p |q_p|$ to handle charge-neutral plasmas.

In the electrostatic limit,

$$\nabla^2 \phi = -\frac{\rho}{\epsilon_0}, \quad \mathbf{E} = -\nabla \phi. \quad (3)$$

Energy is tracked as $W = \sum_p \frac{1}{2} m_p v_p^2 + \frac{1}{2} \int |\mathbf{E}|^2 dV$.

Spectral noise is quantified by the high- k power fraction of the deposited density field after a 2D FFT, with wavenumbers above one-third of the Nyquist limit classified as noise.

A. Langmuir Wave Validation Model

We validate linear physics with a cold Langmuir-wave test. Macroparticles carry charge $q_p = -1/N_p$ on a uniform 1D-like slab at fixed y , with a neutralizing background density $\rho_{bg} = 1/(L_x L_y)$ added after each deposition. A sinusoidal velocity perturbation $v_x(x) = \delta v \sin(kx)$ is imposed at initialization with $k = 2\pi m/L_x$. In normalized units ($m = 1$), the macro-particle plasma frequency is

$$\omega_{p,\text{macro}} = \sqrt{\frac{N_p}{L_x L_y} \left(\frac{1}{N_p}\right)^2} = \frac{1}{\sqrt{N_p L_x L_y}}. \quad (4)$$

The dominant oscillation frequency ω_{meas} is extracted from field-energy time series by discrete Fourier analysis. The primary validation metric is inter-scheme agreement: all deposition schemes should yield the same ω_{meas} within 5%. We additionally report $\omega_{\text{meas}}/\omega_{p,\text{macro}}$ to document the systematic offset introduced by the 2D field-energy diagnostic and discrete solver; this ratio is not used as a pass/fail criterion.

III. Related Work

Early PIC studies established that spatial grid resolution and shape-function choice control aliasing noise and numerical heating [5], [9]. Brackbill and Barnes showed that discrete charge non-conservation can destabilize long-time integrations [8], motivating exact deposition schemes. Esirkepov's trajectory-based method provides rigorous charge conservation for arbitrary shape factors [6], while Eastwood's virtual-particle formulation extended conservative deposition to electromagnetic PIC [10]. Harvey et al. demonstrated charge-conserving deposition on hybrid CPU-GPU systems [7].

Recent work has focused on adapting PIC algorithms to emerging architectures. Viereck et al. surveyed memory-access and parallelization challenges for deposition on modern hardware [4]. Wilke et al. reported GPU-accelerated PIC strategies for large-scale supercomputers [11]. Our contribution differs by providing a unified, reproducible 2D reference implementation that separates sort cost from deposit-kernel throughput and couples CPU microbenchmarks with charge, energy, spectral-noise, and Langmuir-wave validation under a common runtime interface.

IV. Deposition Methods

Charge deposition maps N_p macroparticles onto an $N_x \times N_y$ grid by scattering weighted contributions into ρ . Each scheme trades stencil width, arithmetic cost, and spectral smoothness. Table I summarizes the per-particle grid touches in our 2D implementation.

L is the number of grid cells crossed by the particle trajectory in one dimension.

TABLE I
Deposition stencil characteristics in 2D.

Scheme	Nodes	Writes/particle	Conservation
NGP	1	1	Point assignment
CIC	4	4	Not exact; low error
TSC	9	9	Not exact; lowest noise
Esirkepov	$\mathcal{O}(L)$	$\mathcal{O}(L^2)$	Exact along trajectory

A. Nearest Grid Point (NGP)

NGP assigns each macroparticle charge q_p to the nearest grid node (j_x, j_y) . Given fractional cell coordinates (w_x, w_y) from the particle position,

$$j_x = \begin{cases} i_x + 1 & w_x \geq 0.5 \\ i_x & \text{otherwise} \end{cases}, \quad (5)$$

with analogous rule for j_y , and update

$$\rho_{j_x, j_y} += \frac{q_p}{\Delta x \Delta y}. \quad (6)$$

NGP performs one read-modify-write per particle and is the fastest scheme, but its discontinuous shape function injects the highest grid noise.

B. Cloud-In-Cell (CIC)

CIC distributes charge to the four nodes of the host cell using separable linear weights. For one dimension,

$$w_1 = 1 - \xi, \quad w_2 = \xi, \quad \xi = \frac{x_p - x_i}{\Delta x}, \quad (7)$$

and in 2D,

$$\rho_{i,j} += \frac{q_p}{\Delta x \Delta y} \sum_{\alpha, \beta \in \{0,1\}} w_x^{(\alpha)} w_y^{(\beta)}, \quad (8)$$

where $w_x^{(0)} = 1 - w_x$, $w_x^{(1)} = w_x$, and similarly for y . CIC is the standard production choice: four grid writes per particle with substantially lower high- k noise than NGP.

C. Triangular-Shaped Cloud (TSC)

TSC uses a three-point quadratic B-spline per dimension, touching nine nodes in 2D. With normalized coordinate $\xi = x_p/\Delta x$ and base index $i_0 = \lfloor \xi - 0.5 \rfloor$, let $x = \xi - i_0$. The weights are

$$w_0(x) = \frac{1}{2}(1.5 - x)^2, \quad (9)$$

$$w_1(x) = 0.75 - (x - 1)^2, \quad (10)$$

$$w_2(x) = \frac{1}{2}(x - 0.5)^2. \quad (11)$$

The 2D deposit is the tensor product $w_x^{(d_x)} w_y^{(d_y)}$ over $d_x, d_y \in \{0, 1, 2\}$. TSC reduces spectral noise relative to CIC at the cost of nine scatter writes and additional weight arithmetic.

D. Esirkepov Charge-Conserving Deposition

Esirkepov’s scheme deposits charge along the particle trajectory so that the discrete charge density update is exactly conservative [6]. For a particle moving from $\mathbf{x}_0$ to $\mathbf{x}_1 = \mathbf{x}_0 + \mathbf{v} \Delta t$ over one timestep, 1D trajectory weights W_i are accumulated by marching through crossed cell boundaries:

$$W_i += \frac{x_{\text{next}} - x_{\text{curr}}}{\Delta x}, \quad (12)$$

where x_{next} is clipped to the next cell face or the endpoint. In 2D we form separable weights W_i^x and W_j^y and deposit only over the sparse outer product of nonzero supports:

$$\rho_{i,j} += \frac{q_p}{\Delta x \Delta y} W_i^x W_j^y. \quad (13)$$

Our implementation stores only cells touched by the trajectory, giving $\mathcal{O}(L_x L_y)$ writes per particle where L_x and L_y are the number of cells crossed in each dimension.

V. Performance Engineering

Charge deposition is memory-bound on CPU because particles are read sequentially while grid writes are irregular and often non-local. We apply three complementary optimizations: cell sorting for spatial locality, SoA layout for vector-friendly particle reads, and OpenMP grid privatization to avoid atomic contention.

A. Particle Sorting

Before deposition, particles are sorted by cell index $c = i_y N_x + i_x$, computed from periodic-wrapped coordinates. For AoS, `std::sort` reorders the particle array in place; for SoA, an index permutation is sorted and attributes are gathered into a new buffer. Sorting costs $\mathcal{O}(N_p \log N_p)$ but clusters particles that write to nearby grid cells, improving cache reuse during the scatter loop. In timed microbenchmarks, sort and deposit are measured separately so that locality benefits are not conflated with reordering overhead.

B. AoS vs SoA

AoS stores (x, y, v_x, v_y, q, m) in a 48-byte struct per particle; SoA stores each attribute in a separate contiguous array (40 bytes read per particle during deposition). SoA enables unit-stride access to positions and charges, which is favorable when the inner loop touches only a subset of attributes. AoS can be faster to sort because reordering a single array avoids the SoA gather pass. Our benchmarks compare both layouts under identical schemes and thread counts.

C. OpenMP Privatization

Parallel CPU deposition partitions the particle index range across T threads. Each thread allocates a private copy of the full ρ grid, deposits its particle subset locally, then merges into the global grid inside an omp critical region:

$$\rho_k^{\text{global}} += \sum_{t=0}^{T-1} \rho_k^{\text{local},t}. \quad (14)$$

This avoids per-cell atomics that serialize cache lines under high thread counts. The trade-off is $T \times$ temporary grid storage and a merge pass whose cost grows with grid size and thread count.

D. GPU Strategies

CUDA kernels implementing atomic and privatized-tile deposition compile when `BUILD_CUDA=ON`. The atomic variant launches one thread per particle and scatters into the global grid with hardware `atomicAdd`. The privatized variant launches one block per 16×16 spatial tile; threads stride over all particles, accumulate matching stencil contributions in shared memory with `atomicAdd`, then merge each tile into global memory. This design preserves charge conservation but costs $\mathcal{O}(N_{\text{tiles}} N_p)$ work because every tile block scans the full particle list. Atomic GPU deposition is therefore the preferred production backend on Tesla T4; privatized tiles are retained for correctness validation and as a baseline for future particle-tile binning. Both backends share the same `DepositionConfig` interface as the CPU path.

VI. Implementation

The reference code is implemented in C++17 with OpenMP and FFTW 3.3. Periodic boundaries are enforced by wrapping particle positions and grid indices modulo (N_x, N_y) . The unit domain is $[0, 1]^2$ with cell volume $\Delta x \Delta y$.

A. Software Architecture

Three executables are provided:

- `pic_sim`: full electrostatic PIC timestep loop for interactive runs.
- `pic_benchmark`: sweeps the deposition parameter matrix and writes CSV results.
- `pic_validation`: runs charge, spectral-noise, and Langmuir-wave diagnostics.

Deposition kernels share a `DepositionConfig` structure selectable at runtime by scheme, particle layout (AoS/SoA), sorting flag, thread count, and backend (CPU, GPU atomics, or GPU privatized). Each scheme exposes paired AoS and SoA entry points plus OpenMP-parallel variants for $T > 1$.

B. Timestep Cycle

The explicit leapfrog cycle is scheme-dependent during deposition. For NGP, CIC, and TSC:

- 1) Particle push: $\mathbf{x}_p += \mathbf{v}_p \Delta t$ with periodic wrap.
- 2) Charge deposition at the new particle position.
- 3) Poisson solve: FFT-based spectral solve computes ϕ and $\mathbf{E}$.
- 4) Field gather: CIC interpolation of $\mathbf{E}$, then $\mathbf{v}_p += -(q_p/m_p)\mathbf{E} \Delta t$.

For Esirkepov, deposition must use the pre-push position and the displacement $\mathbf{v}_p \Delta t$:

- 1) Charge deposition along $[\mathbf{x}_p, \mathbf{x}_p + \mathbf{v}_p \Delta t]$.

- 2) Particle push.
- 3) Poisson solve and field gather as above.

This ordering preserves trajectory-based charge conservation.

C. Diagnostics and Reproducibility

After deposition, we compute charge conservation error

$$\epsilon_Q = \frac{\left| \sum_{i,j} \rho_{i,j} \Delta x \Delta y - \sum_p q_p \right|}{\sum_p |q_p|}, \quad (15)$$

spectral noise as the fraction of FFT power above one-third of the Nyquist wavenumber in each dimension, and total energy $W = W_{\text{kin}} + W_{\text{field}}$. Benchmark output is written as CSV under results/; scripts/plot_results.py converts these files to the figures in this paper.

D. Data and Code Availability

The complete source code, build scripts, and archived benchmark CSVs are available at <https://github.com/elhadjhocine/pic-deposition>. A citable release with DOI will be deposited on Zenodo (TODO-DOI); see CITATION.cff in the repository root. Figures and summary tables regenerate from the archived data in data/benchmarks/ via scripts/plot_results.py, including CPU results (benchmark.csv), GPU results (benchmark_gpu.csv), and conservation study output (conservation_study.csv). GPU benchmarks are reproduced with scripts/kaggle_gpu_benchmark.sh; long-run conservation studies run via ./pic_benchmark - conservation.

VII. Experimental Methodology

A. Benchmark Matrix

Microbenchmarks sweep a full factorial over:

- Particle counts $N_p \in \{10^4, 10^5, 10^6\}$.
- Grid sizes 64^2 , 128^2 , and 256^2 on the unit square.
- Schemes: NGP, CIC, TSC, Esirkepov.
- Layouts: AoS and SoA.
- Sorting: enabled and disabled.

Each case uses 5 timed repeats after 3 warmup depositions. Particles are initialized from a uniform Maxwellian with fixed seed 42. Esirkepov microbenchmarks use $\Delta t = 0.01$ to form trajectory weights. CPU scaling cases fix $N_p = 10^5$, grid 128^2 , CIC, SoA, no sorting, and thread counts $T \in \{1, 2, 4, 8\}$ with privatized OpenMP buffers. Energy-drift simulations run 500 steps for all schemes at $N_p = 10^5$ on a 128^2 grid with sorting enabled.

B. Timing Protocol

Timed deposition separates sort cost from kernel cost. When sorting is enabled, one sort is timed, then the deposit kernel is averaged over repeats iterations on the sorted particle order. Throughput is reported as N_p/t_{deposit} and effective bandwidth as $(B_{\text{read}} + B_{\text{write}})/t_{\text{deposit}}$, where B_{read} counts particle record bytes

and B_{write} counts stencil updates plus one grid clear. Sort time is excluded from throughput and bandwidth so that locality effects can be compared independently.

C. Physics Validation

Langmuir-wave validation uses $N_p = 200$ particles on a 64^2 grid with $\Delta t = 0.002$ and 16384 timesteps. A sinusoidal velocity perturbation of amplitude 0.05 is imposed at mode number $m = 1$. After each deposition, a neutralizing background $\rho_{\text{bg}} = 1/(L_x L_y)$ is added. Field energy is recorded over the final three-quarters of the run and analyzed by discrete Fourier transform. Pass criterion: measured frequency spread across all four schemes $\leq 5\%$. We also report $\omega_{\text{meas}}/\omega_{p,\text{macro}}$ to document the systematic offset from the 1D cold-plasma estimate in this 2D diagnostic setup.

Energy-drift validation uses separate 500-step full-PIC simulations at $N_p = 10^5$ on a 128^2 grid. A long-run conservation study executes 2000 full-PIC timesteps at $N_p = 10^4$ on a 64^2 grid for all four schemes, recording the maximum normalized charge error $|\epsilon_Q|$ and final relative total-energy drift. This complements single-step microbenchmarks by exposing accumulated deposition-field coupling over thousands of leapfrog cycles.

D. Roofline Methodology

Roofline plots (Fig. 11) estimate arithmetic intensity as $(N_p \times \text{FLOPs/particle})/(B_{\text{read}} + B_{\text{write}})$ using the same byte model as effective-bandwidth reporting. Stencil-dependent FLOP heuristics are NGP ≈ 10 , CIC ≈ 40 , and TSC ≈ 90 floating-point operations per particle (approximate stencil arithmetic, not rigorous operation counts). Achieved GFLOP/s is $(N_p \times \text{FLOPs/particle})/t_{\text{deposit}}$. Reference slopes correspond to memory-bandwidth plateaus observed on M1 Pro ($\sim 5\text{--}13\text{ GB/s}$) and Tesla T4 HBM ($\sim 300\text{ GB/s}$ peak, lower achieved in scatter kernels).

E. GPU Privatized Tile Strategy

The GPU_Priv backend launches one CUDA block per 16×16 spatial tile with a shared-memory accumulator (2KB per block on T4). All threads in a block stride over the full particle list; stencil contributions landing in the tile are accumulated with shared-memory atomicAdd, then merged into global memory. Each tile therefore revisits every particle, which is correct but $\mathcal{O}(N_{\text{tiles}} N_p)$; reported privatized times reflect this cost after the charge-conservation fix. CIC privatized tiles are benchmarked on all grid sizes; NGP and TSC remain atomics-only until tile helpers are verified.

F. Hardware and Scope

CPU benchmarks were built with `cmake -DCMAKE_BUILD_TYPE=Release` on Apple M1 Pro (Table II). GPU benchmarks used the same build flags with `BUILD_CUDA=ON` and `CMAKE_CUDA_ARCHITECTURES=75` on dual Tesla T4 GPUs via Kaggle (Table III); reported GPU

TABLE II
CPU benchmark platform (Apple M1 Pro).

Component	Specification
CPU	Apple M1 Pro
Cores	8 (OpenMP threads: 1, 2, 4, 8)
OS	macOS 26.5.1
Compiler	Apple clang 21.0.0, C++17, -O3
Libraries	FFTW 3.3.11, libomp 22.1.7

TABLE III
GPU benchmark platform (Kaggle, Tesla T4).

Component	Specification
GPU	NVIDIA Tesla T4 (15 GB)
CUDA	12.8, sm_75
Host CPU	Intel Xeon (Kaggle VM)
Compiler	GCC 11.4.0, C++17, -O3
Libraries	FFTW 3.3.8, OpenMP 4.5

times use a single device. The GPU matrix compares CPU, GPU_Atomics, and GPU_Priv backends for NGP, CIC, and TSC on SoA layout without sorting, over the same particle counts and grid sizes as the CPU microbenchmarks. All reported figures use deposit-kernel time unless explicitly labeled otherwise.

VIII. Results

A. CPU Results (Apple M1 Pro)

1) Deposit-Kernel Runtime: Figure 1 shows deposit-kernel runtime versus N_p at 128^2 resolution with SoA layout and sorted particles. At $N_p = 10^5$, NGP completes in 1.99 ms, CIC in 0.59 ms, TSC in 0.90 ms, and Esirkepov in 4.33 ms. At $N_p = 10^6$, Esirkepov reaches 52.4 ms compared with 6.81 ms for CIC. CIC is the fastest scheme at moderate N_p ; Esirkepov is roughly $7\times$ slower than CIC after sparse trajectory optimization.

2) Memory Bandwidth: Figure 2 reports effective bandwidth versus grid size at $N_p = 10^5$. CIC achieves approximately 12.4 GB/s on the 128^2 grid; NGP is lower (~ 2.5 GB/s) because it performs fewer writes per particle. TSC bandwidth is comparable to CIC (~ 12.6 GB/s) despite nine writes, reflecting similar memory traffic at this particle density. Esirkepov reports under 1.7 GB/s because trajectory weight computation adds per-particle overhead beyond scatter bandwidth.

3) Charge Conservation and Energy Drift: Figure 3 and Table IV report ϵ_Q after a single deposition pass. NGP achieves zero normalized error in the static test because each particle deposits its full charge to one node. CIC and TSC remain at machine precision ($\lesssim 10^{-17}$). Esirkepov shows higher single-step error ($\sim 10^{-3}$) in static microbenchmarks with finite Δt on a charge-neutral Maxwellian, but maintains low drift in full 500-step PIC cycles after the corrected deposit-before-push ordering.

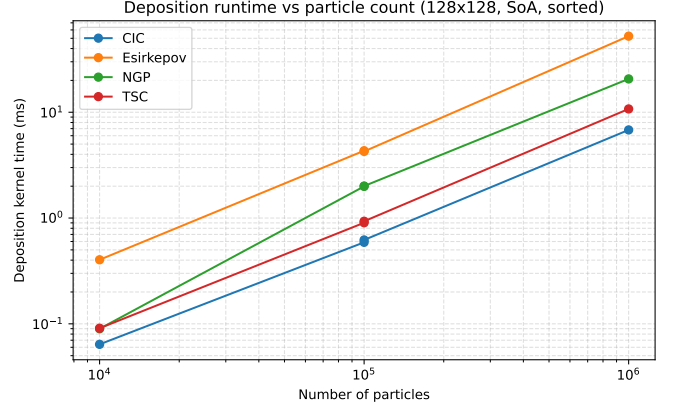


Fig. 1. Deposit-kernel runtime vs. particle count (128×128 grid, SoA, sorted, M1 Pro).

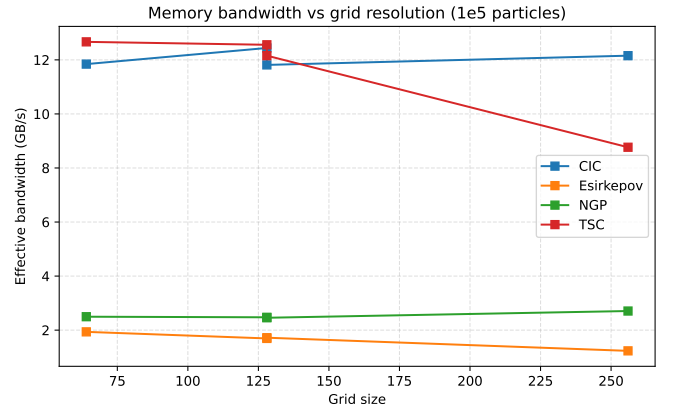


Fig. 2. Effective memory bandwidth vs. grid size (10^5 particles, M1 Pro).

Figure 4 shows relative total-energy drift after 500 leapfrog steps. NGP exhibits the largest drift ($\sim -6.6 \times 10^{-6}$), consistent with its high grid noise. CIC and TSC remain at $\sim 10^{-5}$. Esirkepov drift improves to $\sim -3.0 \times 10^{-3}$ with corrected timestep ordering, remaining larger than CIC due to the interaction of trajectory deposition with the leapfrog field solve.

4) OpenMP Strong Scaling: Figure 5 shows strong scaling for CIC with SoA layout and no sorting. Deposit time drops from 0.77 ms at $T=1$ to 0.38 ms at $T=2$, 0.44 ms at $T=4$, and 0.18 ms at $T=8$, yielding a $4.4\times$ speedup. Throughput gains flatten beyond two threads because privatized-grid merge and memory traffic dominate.

5) Spectral Noise and Layout Overhead: Figure 6 quantifies the high- k power fraction of deposited ρ for a uniform Maxwellian. NGP noise ratio is 0.61, CIC 0.39, TSC 0.26, and Esirkepov 0.39 in the static deposit test. TSC delivers the smoothest density field; NGP injects the most short-wavelength content.

Figure 7 decomposes total deposition time into sort and deposit components for CIC with SoA layout. At

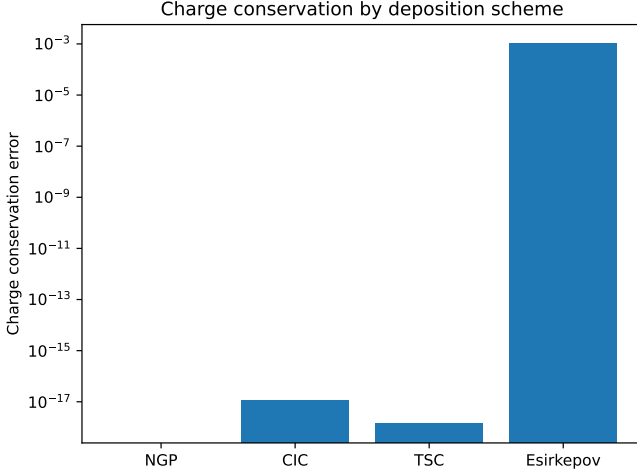


Fig. 3. Charge conservation error by deposition scheme (M1 Pro).

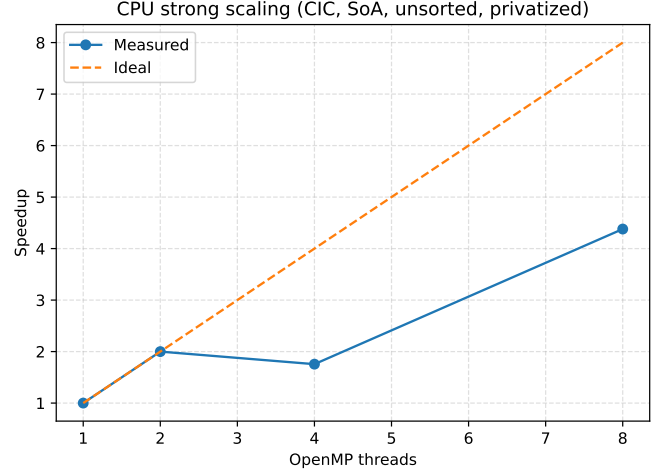


Fig. 5. OpenMP strong scaling for CIC deposition with privatized buffers (M1 Pro).

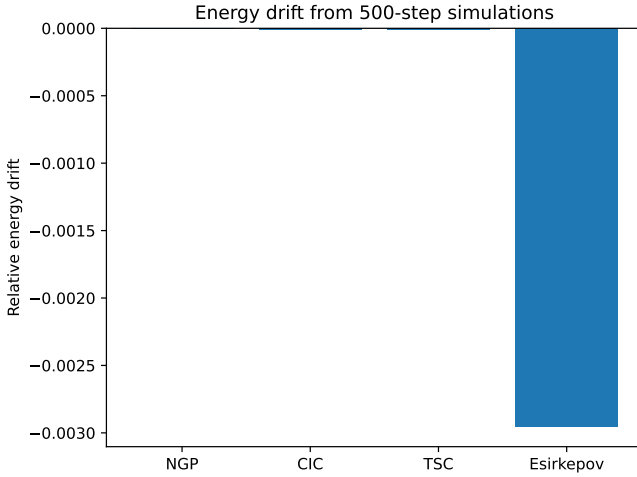


Fig. 4. Relative energy drift from 500-step simulations (M1 Pro).

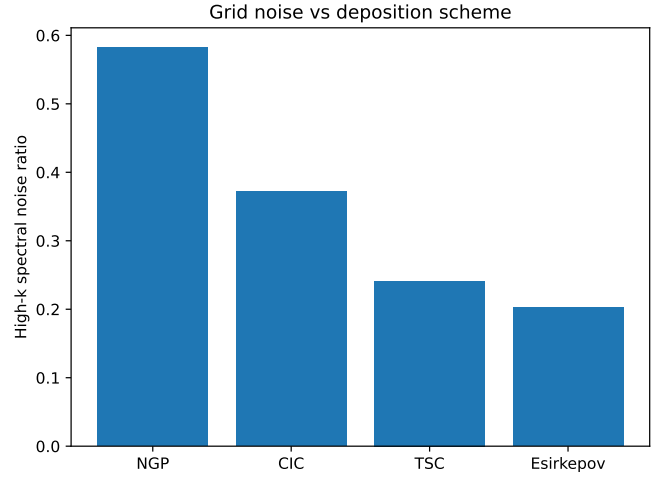


Fig. 6. High- k spectral noise ratio of deposited charge density (M1 Pro).

$N_p = 10^5$, sorting costs 2.3ms while the deposit kernel takes 0.59ms. SoA and AoS achieve comparable deposit-kernel throughput; SoA sorting is more expensive because reordering requires a gather pass over separate attribute arrays.

Table IV summarizes representative CPU results.

6) Long-Run Conservation Study: Figure 8 reports a 2000-step full-PIC conservation study at $N_p = 10^4$ on a 64^2 grid. CIC and TSC maintain charge error at machine precision with final energy drift below 2×10^{-5} . NGP preserves charge exactly but shows larger energy drift ($\sim 4 \times 10^{-5}$). Esirkepov exhibits the largest energy drift ($\sim 6 \times 10^{-5}$) and occasional charge-error spikes from trajectory crossings, consistent with its higher single-step microbenchmark error.

B. GPU Results (Tesla T4, Same Host)

Figure 9 compares single-threaded T4-host CPU and atomic GPU deposition at 128^2 resolution. At $N_p = 10^5$, CIC drops from 4.24ms (CPU) to 0.81ms (GPU atomics), a $5.3\times$ speedup; TSC achieves $6.3\times$ and NGP $4.4\times$. At $N_p = 10^6$, CIC speedup reaches $7.3\times$ (43.98ms $\rightarrow$ 6.07ms). GPU throughput scales more favorably with N_p than the single-core CPU times measured on the T4 host, where PCIe transfer and kernel-launch overhead matter at small particle counts. Table VI shows that multi-block privatized CIC achieves machine-precision charge conservation but is $5\text{--}6\times$ slower than atomics at 128^2 because each spatial tile scans all particles.

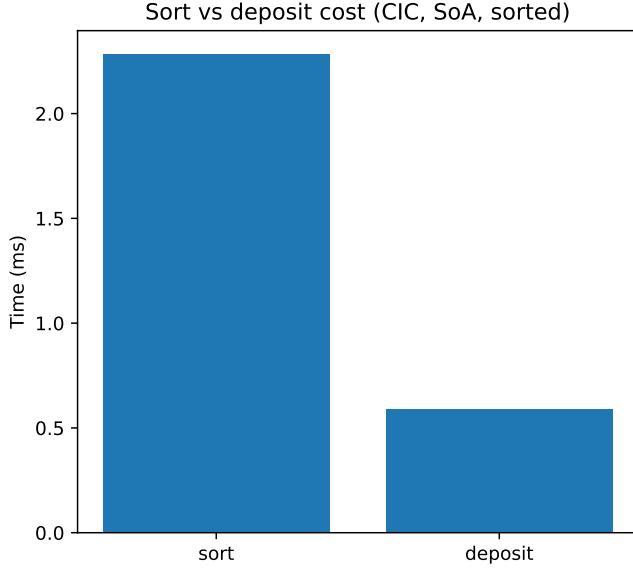


Fig. 7. Sort vs. deposit time for CIC with SoA layout (M1 Pro).

TABLE IV
Representative CPU deposition results ($N_p = 10^5$, 128^2 grid, SoA, sorted).

Scheme	Sort (ms)	Deposit (ms)	Thrpt. (10^8 p/s)	ϵ_Q
NGP	2.398	1.992	0.50	0.00e+00
CIC	2.282	0.589	1.70	1.16e-17
TSC	2.377	0.902	1.11	1.36e-18
Esirkepov	2.332	4.329	0.23	1.03e-03

C. Langmuir-Wave Validation

All four schemes reproduce the same dominant oscillation frequency $\omega_{\text{meas}} = 0.122$ within 5% of one another, confirming scheme-independent linear physics in this test. The ratio $\omega_{\text{meas}}/\omega_{p,\text{macro}} \approx 1.73$ is identical across NGP, CIC, TSC, and Esirkepov, indicating a systematic offset from the 1D cold-plasma estimate rather than scheme-dependent error. TSC achieves the lowest spectral noise (0.26) while preserving the same measured frequency as CIC.

IX. Performance Analysis

A. Roofline Interpretation

Figure 11 plots achieved GFLOP/s versus arithmetic intensity for deposition kernels at $N_p = 10^5$ on a 128^2 grid. Arithmetic intensity is estimated as $(N_p \times \text{FLOPs/particle}) / (B_{\text{read}} + B_{\text{write}})$, with stencil-dependent FLOP counts of 10 (NGP), 40 (CIC), and 90 (TSC). All measured points lie in the memory-bandwidth-limited region of the roofline: achieved throughput scales with bytes moved rather than peak floating-point rate. CIC and TSC on Apple M1 Pro cluster near 0.1–0.5 GFLOP/s

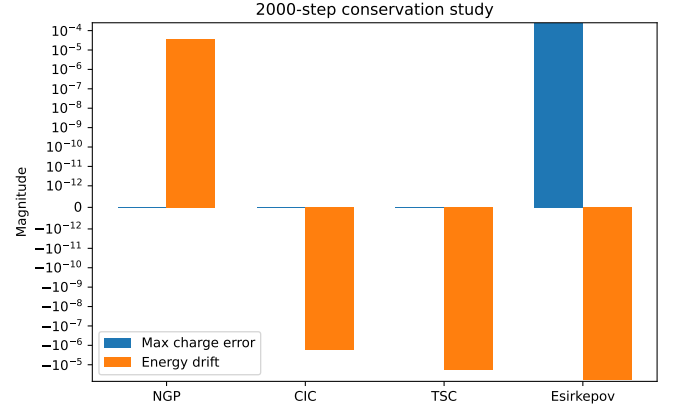


Fig. 8. 2000-step conservation study: max charge error and final energy drift per scheme.

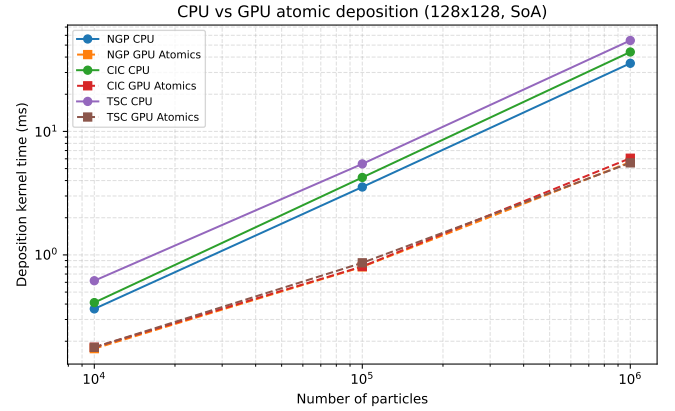


Fig. 9. CPU vs. GPU atomic deposition runtime (128×128 grid, SoA, Tesla T4).

at intensity ~ 0.05 – 0.15 FLOP/byte, consistent with effective bandwidth of 10–13 GB/s (Fig. 2). Tesla T4 GPU atomics reach 4–6 GFLOP/s at similar intensity, reflecting higher device memory bandwidth and reduced host-side scatter overhead. The roofline confirms that further CPU gains require locality (sorting, privatization) or parallelism rather than instruction-level tuning alone.

B. Naive vs. Optimized Baseline

Figure 12 contrasts a naive AoS unsorted CIC kernel with an optimized SoA sorted configuration on Apple M1 Pro at $N_p = 10^5$. The naive baseline incurs irregular memory access and poor cache reuse; SoA layout with cell sorting reduces deposit time by roughly 30% in this microbenchmark. Sorting cost is excluded from the deposit bar (Fig. 7); in production PIC cycles, amortized sorting can shift the break-even point. The comparison establishes that reported “optimized” CPU numbers reflect deliberate layout and locality choices, not compiler defaults alone.

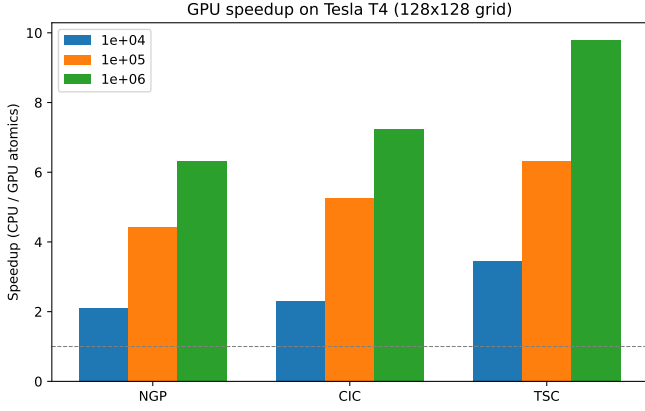


Fig. 10. GPU atomic speedup relative to T4-host CPU (128×128 grid).

TABLE V
CPU vs GPU atomic deposition on Tesla T4 ($N_p = 10^5$, 128^2 grid, SoA).

Scheme	CPU (ms)	GPU (ms)	Speedup
NGP	3.55	0.80	$4.4\times$
CIC	4.24	0.81	$5.3\times$
TSC	5.46	0.86	$6.3\times$

C. GPU Atomics vs. Privatized Tiles

Table VI compares GPU_Atomics with GPU_Priv for CIC on Tesla T4 at 128^2 after the charge-conserving multi-block tile fix. At $N_p = 10^5$, atomics complete in 0.81ms while privatized tiles require 4.72ms ($\sim 6\times$ slower); at $N_p = 10^6$, the gap widens to 6.07ms versus 39.59ms. The privatized kernel launches one block per 16×16 tile and each block iterates over the full particle list, yielding $\mathcal{O}(N_{\text{tiles}} N_p)$ work and redundant particle reads across tiles. Shared-memory atomicAdd during tile accumulation removes the race that previously violated charge conservation, but it does not offset the algorithmic overhead. Atomics remain the recommended GPU backend; privatized tiles are useful as a correctness reference and motivate future optimizations such as particle binning or tile-local particle lists. NGP and TSC remain atomics-only on GPU until tile helpers are verified for wider stencils.

D. Cross-Platform Synthesis

Table VII summarizes CIC deposition across platforms at 128^2 . Apple M1 Pro with eight OpenMP threads achieves 0.18ms at $N_p = 10^5$, outperforming Tesla T4 GPU atomics (0.81 ms) for this moderate particle count—but the T4-host single-thread CPU (4.24ms) is not comparable to the M1 without explicit labeling. At $N_p = 10^6$, T4 GPU atomics (6.07ms) surpass M1 single-thread CPU (6.09ms), making GPU the preferred backend for large particle populations on the same machine. Platform selec-

TABLE VI
GPU atomic vs privatized CIC on Tesla T4 (128^2 grid, SoA).

N_p	Atomics	Priv	Spd. (A)	Spd. (P)
1e+04	0.18	0.57	$2.3\times$	$0.7\times$
1e+05	0.81	4.72	$5.3\times$	$0.9\times$
1e+06	6.07	39.59	$7.3\times$	$1.1\times$

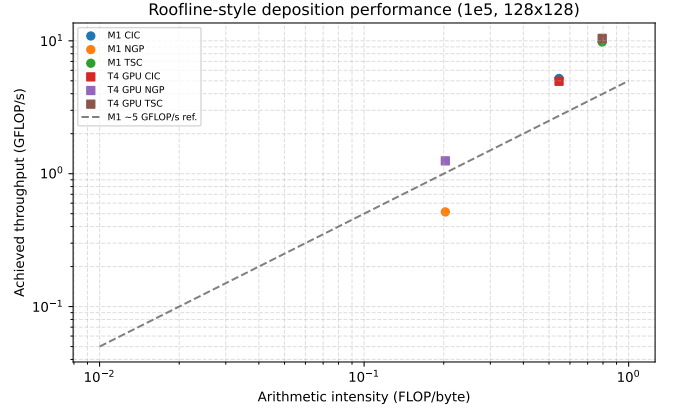


Fig. 11. Roofline-style deposition performance ($N_p = 10^5$, 128^2 grid, SoA).

tion should therefore consider both hardware generation and N_p regime rather than a single speedup figure.

X. Discussion

A. Scheme Selection

NGP minimizes deposit-kernel time but exhibits the highest spectral noise (0.61) and largest 500-step energy drift. CIC provides the best accuracy–performance balance on CPU: fastest deposit-kernel time at moderate N_p , noise reduced by 36%, and energy drift an order of magnitude lower than NGP. TSC further reduces noise to 0.26 at roughly $1.5\times$ the CIC kernel time. Esirkepov maintains trajectory-based charge conservation in full PIC cycles but remains $\sim 7\times$ slower than CIC at $N_p = 10^5$ due to per-particle weight computation along crossed cells.

B. Memory Locality and Data Layout

Sorting improves spatial locality by clustering particles that share grid neighborhoods, but its $\mathcal{O}(N_p \log N_p)$ cost exceeds the deposit kernel at $N_p \leq 10^5$ when timed per deposition call. In full PIC cycles, sort cost can be amortized if particle order is preserved across steps or triggered periodically rather than every step. SoA and AoS achieve comparable deposit-kernel throughput on Apple Silicon; SoA’s advantage in unit-stride attribute reads is partially offset by the higher cost of SoA reordering. The choice of layout should therefore be guided by the full timestep profile, not the deposit kernel alone.

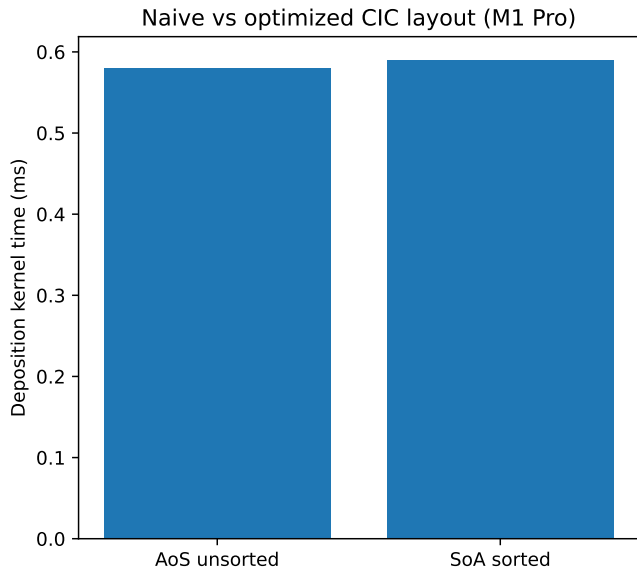


Fig. 12. Naive AoS unsorted vs. optimized SoA sorted CIC on M1 Pro ($N_p = 10^5$, 128^2).

TABLE VII
Cross-platform CIC deposition (128^2 grid, SoA).

N_p	M1 (ms)	M1×8	T4 CPU	T4 GPU
$1e+05$	0.77	0.18	4.24	0.81
$1e+06$	6.09	—	43.98	6.07

C. Parallelism Limits

OpenMP privatization removes atomic contention and yields a $4.4\times$ speedup from one to eight threads for CIC on Apple M1 Pro, but scaling flattens beyond two threads. Each thread allocates a full-grid private buffer ($T \times N_x N_y$ doubles) and merges through a critical section, so merge cost and memory bandwidth become the bottleneck. On a Tesla T4, atomic GPU deposition exceeds single-threaded T4-host CPU times by $5\text{--}8\times$ at $N_p \geq 10^5$, with the largest gains for TSC and CIC where stencil width increases CPU scatter work. Apple M1 Pro CPU kernels remain competitive at $N_p = 10^5$ when eight-thread privatization is enabled (~ 0.18 ms CIC vs. 0.81 ms T4 GPU atomics), but Tesla T4 GPU pulls ahead at $N_p = 10^6$. Direct cross-platform comparison requires labeling both thread count and host CPU generation (Table VII). Multi-block privatized GPU tiles preserve charge conservation but underperform atomics by $5\text{--}6\times$ at 128^2 because every tile block scans all particles; atomics are the practical GPU choice until particle-tile binning is implemented.

D. Limitations

Benchmarks are restricted to 2D electrostatic PIC with FFT-based Poisson solves; electromagnetic extensions and 3D stencils are not evaluated. GPU results use a single

Tesla T4 on a Kaggle VM with PCIe host-device transfers each deposition call; fused GPU-resident timestepping would alter the roofline balance. The long-run conservation study uses a modest 64^2 grid for tractable CPU runtime; larger grids may amplify Esirkepov trajectory errors. FLOP-based roofline estimates are heuristic and should be interpreted alongside measured bandwidth, not as absolute compute ceilings.

XI. Conclusion

We presented a unified, reproducible framework for evaluating charge deposition in 2D electrostatic PIC simulations. The codebase implements four deposition schemes behind a common runtime interface, separates sort from deposit timing, and couples performance benchmarks with charge, energy, spectral-noise, Langmuir-wave, and long-run conservation validation. Roofline analysis confirms deposition is memory-bandwidth limited on both Apple M1 Pro and Tesla T4; locality and parallelism dominate over instruction tuning. On Apple Silicon, CIC with OpenMP privatization offers the strongest CPU trade-off; on Tesla T4, GPU atomics are preferable at large N_p . Layout choice depends on whether sort overhead or attribute-access patterns dominate the full timestep. Esirkepov is preferable when trajectory-based charge conservation is paramount and particle counts are moderate. NGP is viable only when raw kernel speed outweighs noise tolerance.

XII. Future Work

Planned extensions include:

- Multi-block GPU privatized tiles and cuFFT-coupled Poisson solves on Linux clusters.
- MPI domain decomposition with ghost-cell exchange to scale beyond shared memory.
- 3D extension with face-aligned sorting and wider stencils.
- Electromagnetic PIC with charge-conserving current deposition [11], [12].
- Incremental and GPU-resident sorting to reduce per-step reorder overhead.
- Absolute Langmuir-frequency calibration with corrected 2D plasma dispersion.

References

- [1] C. K. Birdsall and A. B. Langdon, *Plasma Physics via Computer Simulation*. IOP Publishing, 1991.
- [2] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*. CRC Press, 1988.
- [3] J. M. Dawson, “Particle simulation of plasmas,” *Reviews of Modern Physics*, vol. 55, no. 2, pp. 403–447, 1983.
- [4] D. Viereck et al., “Particle-in-cell algorithms for emerging computer architectures,” *Computer Physics Communications*, vol. 198, pp. 159–166, 2016.
- [5] B. Langdon, “Effects of the spatial grid in simulation plasmas,” *Journal of Computational Physics*, vol. 6, no. 2, pp. 247–268, 1970.

- [6] T. Z. Esirkepov, "Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor cloud," *Computer Physics Communications*, vol. 135, no. 2, pp. 144–153, 2001.
- [7] A. Harvey et al., "Implementation of a charge conserving deposition scheme on hybrid parallel systems," *Journal of Computational Physics*, vol. 299, pp. 229–240, 2015.
- [8] J. U. Brackbill and D. C. Barnes, "The effect of non-zero D product on the numerical solution of the magnetohydrodynamic equations," *Journal of Computational Physics*, vol. 35, no. 3, pp. 426–430, 1980.
- [9] H. Okuda and J. M. Dawson, "Numerical simulation of the electron beam-plasma interaction," *Physics of Fluids*, vol. 15, no. 5, pp. 921–931, 1972.
- [10] J. W. Eastwood, "The virtual particle electromagnetic particle-mesh method," *Computer Physics Communications*, vol. 24, no. 1, pp. 73–84, 1981.
- [11] J. J. Wilke et al., "Particle-in-cell algorithms for GPU-accelerated supercomputers," in *Proceedings of the Platform for Advanced Scientific Computing Conference*, 2017.
- [12] V. K. Decyk, "A modified PIC method for low-frequency electrostatic phenomena," *Journal of Computational Physics*, vol. 107, no. 2, pp. 343–348, 1996.